Part II — Scoping & Trade-offs Write-Up

Priorities — what we built first, and what we left out

Built, in order:

1. **Correct single-document edits** — insert, replace, and delete, targeted either by exact text with an occurrence number ("the 3rd 'Delaware'") or by character position. Every edit is versioned: each change appends a new version rather than overwriting, so the document's history is never lost.
2. **Preview before commit** — the same edit logic runs in a dry-run mode that returns the before/after text without writing anything, so the UI can show an exact diff, with a separate accept step to commit it. Editing binding agreements demands seeing an edit's exact effect before it's permanent; true for single and bulk edits alike.
3. **Search** — full-text search (indexed, not a linear scan) across all documents or scoped to one, returning ranked results with highlighted snippets. How the team finds the contracts that need a redline before deciding what to change.
4. **Bulk operations** — for example, "swap this clause in 200 templates". Documents are targeted by an explicit ID list or by the same search query used above; the edit and preview/commit step reuse the single-document logic rather than duplicating it, so a bulk change behaves identically to a manual edit, just applied across many documents in one request. Built last since it's less crucial than the other pieces and reuses functions from them.

Deliberately left out (and why):

- **Word/PDF upload** — contracts likely arrive as files, but I left it out because importing them doesn't change the edit/search core.
- **Undo / revert to a prior version** — every edit is already stored as a version, so the data to revert exists; there just isn't a button/endpoint for it yet.
- **Fuzzy string matching** — for legal edits, an exact match with an explicit occurrence number is more predictable than fuzzy matching, which needs a defined tolerance we'd want to validate against real contracts before shipping.

Assumptions

- **Documents are plain text**, not formatted files — no Word formatting, tables, or images. Fastest path to a working loop; likely not sufficient for how contracts actually arrive today.
- **Documents are modest in size** — kilobytes to a few megabytes (a contract, not a 100MB+ archive). This is what makes the in-memory edit approach the right call.
- **A single redline request carries a handful to a few dozen changes**, not thousands — changes are applied in order against a small, bounded list.
- **Single active editor per document** — concurrent edits aren't detected; last save wins silently.
- **An occurrence number is an acceptable way to disambiguate** repeated text ("the 3rd 'Delaware'") rather than the tool guessing which one is meant. Assumes hundreds of documents — a team's active template set, not a company-wide archive.

Open Questions — before building the next version

- **Is there specific "risky" or non-standard language to flag for?** Search finds where a term is — it doesn't yet judge a clause as unusual. We'd want concrete examples of what counts as risky (a missing clause? non-standard wording of a standard one? an out-of-range dollar figure?) before proposing how to surface it.
- **Does confidentiality require per-client/matter access control?** Determines whether "no accounts / anyone can see everything" is acceptable for v2 or a blocker, and whether tenants need hard data isolation.
- **What format do contracts actually arrive in?** Word with tracked changes, a scanned PDF, and plain text each imply a different next step — anywhere from a straightforward import step to a larger rework of the document model.
- **How large do documents get, and how many are edited at once?** Sets whether current performance headroom is enough or the storage approach needs to change.

Where It Breaks

Against real customer data:

- Word/PDF files can't be edited or searched until converted to plain text — no import step exists today.

At scale:

- Each document loads fully into memory to edit — fine to ~10MB; a much larger document would need a streaming approach.

- Formatting-dependent structure (tables, numbered-list formatting, signature images) is invisible to the tool; only visible text is edited/searched.
- Two people editing the same contract at once isn't detected — last write wins.
- A single SQLite database is right for one team, not for serving many teams/organizations concurrently — a real but well-understood migration to a client-server database.